The Journal of Open Source Software

Kigali Sim: Open source simulation toolkit for modeling substances and policies related to the Montreal Protocol

A Samuel Pottinger [1¶], **Balaji Natarajan** [2], **Magali de Bruyn** [1], **and Ciera Martinez** [1]

1 Schmidt Center for Data Science and Environment, University of California, Berkeley, California, United States of America **2** Secretariat of the Multilateral Fund for the Implementation of the Montreal Protocol, United Nations Environment Programme ¶ Corresponding author

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 04 November 2025
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

Summary

Co-created with over a dozen countries and organizations, Kigali Sim offers reusable stock and flow modeling of substances controlled under the Montreal Protocol. Specifically, this open source platform focuses on the Kigali Amendment which reduces hydrofluorocarbons that contribute to climate change through warming potentials up to thousands of times greater than carbon dioxide, aiding both atmospheric science and policy development. Furthermore, supporting a diverse community with heterogeneous programming expertise, this parallelized toolkit interoperates visual no-code rapid model development and a domain-specific language (DSL), democratizing access to advanced computational tools. Running via WebAssembly (WASM) or Java Virtual Machine (JVM), Kigali Sim offers portable, private, and rigorous simulation in support of history's most successful international environmental treaty.

Statement of need

Signed by all UN member states, the Montreal Protocol successfully phased out 99% of ozone-depleting substances (World Meteorological Organization, 2022). Already the most successful international environmental treaty (McKenzie et al., 2019), its ambitious 2016 Kigali Amendment extends this multilateral framework to hydrofluorocarbons which contribute significantly to climate change (Birmpili, 2018; Velders et al., 2009). However, related research and policy analysis involves considering complex economic, technological, and policy interactions (MLF Staff, 2024), typically through closed source ad-hoc models. National Ozone Units (NOUs) and others who possess deep domain knowledge may work with limited resources (UNEP OzonAction, n.d.) and the current lack of an open source domain-oriented toolkit for holistic stock and flow simulation may inhibit transparent accessible modeling (Augspurger et al., 2023).

Kigali Sim provides the first open source reusable full lifecycle toolkit addressing Montreal Protocol-controlled substances. Consulting over a dozen countries and supporting organizations, a community of on-the-ground scientists, analysts, and policy-makers inform its flexible design. With either code or UI-based editing, it simulates emissions, energy, substance consumption, equipment populations, trade, and policy either in browser or JVM while following treaty conventions (Multilateral Fund Secretariat, 2024).

Design

With ongoing active Article 5 country (United Nations, 1987) use, Kigali Sim spans NOUs, Implementing Agencies, analysts, researchers, project implementers, and other organizations. Its dual-interface design supports domain experts in atmospheric science and environmental policy with varied programming expertise.

Simulation engine

Some countries and supporting organizations work from trade records and industry surveys while others from bank estimates or observed emissions. The engine propagates user-provided values through substance flows and lifecycles to estimate the unmeasured quantities. It then layers complex policy interventions such as permitting and recycling on top of the "business as usual" scenario.

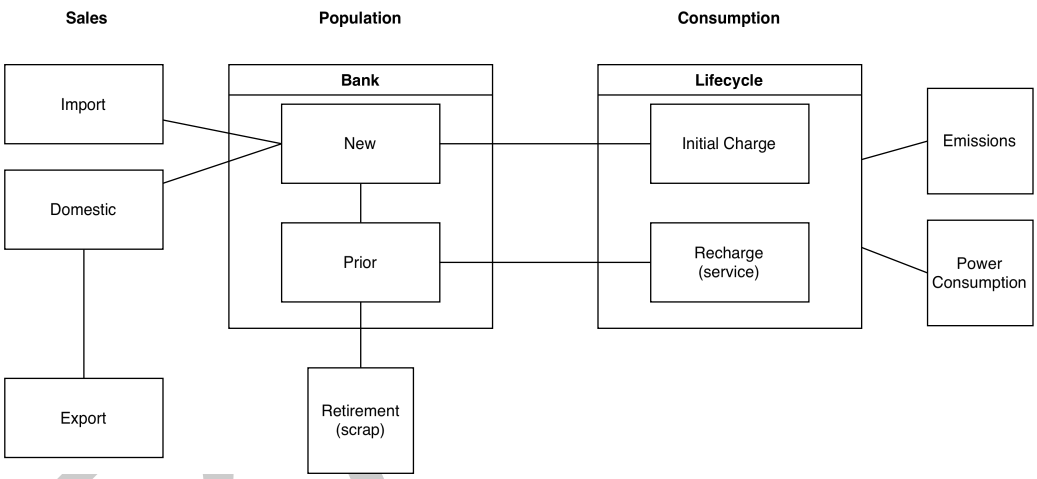

Figure 1: Diagram showing data flow through Kigali Sim simulation engine from input data (trade records, bank estimates, or equipment surveys) through stock and flow calculations to output metrics.

Dual-Interface Design

We find many Kigali Sim users do not identify as programmers and, reflecting empirical findings that domain experts with "limited programming knowledge" benefit from domain-specific languages (Schindler & Cabot, 2022), our software progresses from a UI-based editor to DSL code-based authoring. In all cases, simulations execute locally, ensuring privacy.

UI-based authoring

The UI-based point-and-click editor familiarizes the user with Kigali Sim's concepts, creating a loop-based design (A. Pottinger et al., 2025; Skills, 2020) where small GUI-based changes automatically translate to code run for immediate feedback. Note that the web interface first progressively exposes functionality through sequenced disclosure (Brown, 2015; Hayashida, 2012; A. S. Pottinger & Zarpellon, 2023) as an on-ramp into a more open design (Brown, 2017; A. S. Pottinger et al., 2023).

The Journal of Open Source Software

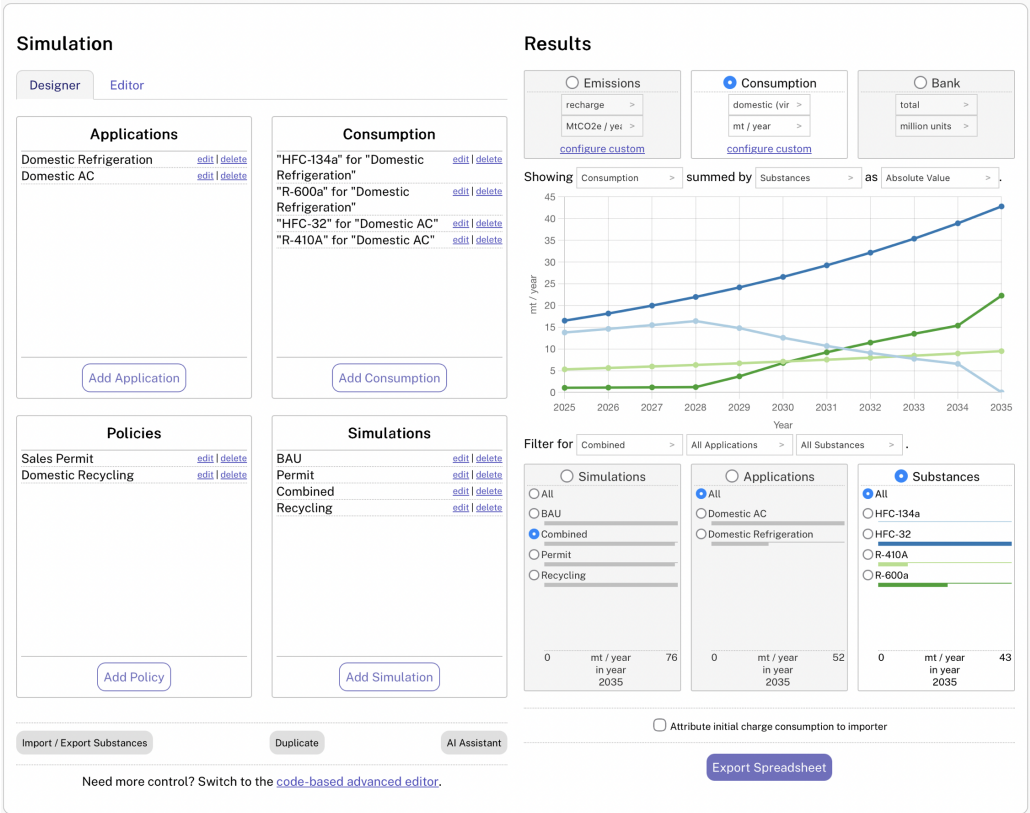

Figure 2: Screenshot of the UI-based editor modifying an example simulation.

Code-based authoring

We find some simulations' complexity cumbersome in UI-based authoring. Therefore, our QubecTalk domain-specific language facilitates expression of complex Montreal Protocol simulations in human-readable syntax inspired by but distinct to HyperTalk (Wheeler, 2004). Mirrored by the UI-editor, QubecTalk speaks in treaty terminology: directly translating terms of art like `cap import to 85% during years 2025 to 2035` into simulations. This also supports uncertainty quantification, conditional logic, and policy stacking. With optional AI assistant compatibility via the llms.txt specification (Answer.AI, 2024), users may author scripts in a web-based programming portal (Appleton, 2022) or third-party editors with direct JVM invocation.

Flexibility

As diverse methodologies measure stocks, Kigali Sim provides automated unit conversions including those which depend on equipment properties. Additionally, most simulation code can be modified either by the UI-based editor or the code-based editor where changes in one reflect in the other, attempting to bridge preferences and skill sets. Indeed, many community collaborators who identify as non-programmers report starting in the UI-editor but transition to code.

Limitations and future work

Leaving the following for future work, Kigali Sim:

- Can model energy consumption but only estimates direct emissions[1].

[1] Users may may manually calculate indirect emissions using exported and energy mix data (Ang et al., 2016).

- Applies treaty trade attribution but will only attribute charge prior to equipment sale entirely to either the importer or exporter[2].
- Complex features like formula-based variable servicing / retirement hazard rates may require code-based authoring.

It will receive updates as official guidance changes in the future (Multilateral Fund Secretariat, 2024).

Implementation

Kigali Sim runs browser-based via WASM or via JVM (Konshin, 2024). Without transmitting simulations to external servers, both enable privacy-preserving zero-installation parallel local computation with BigDecimal (Friesen, 2024). ANTLR (Parr, 2013) parses QubecTalk code in both Java and ECMAScript / JavaScript[3].

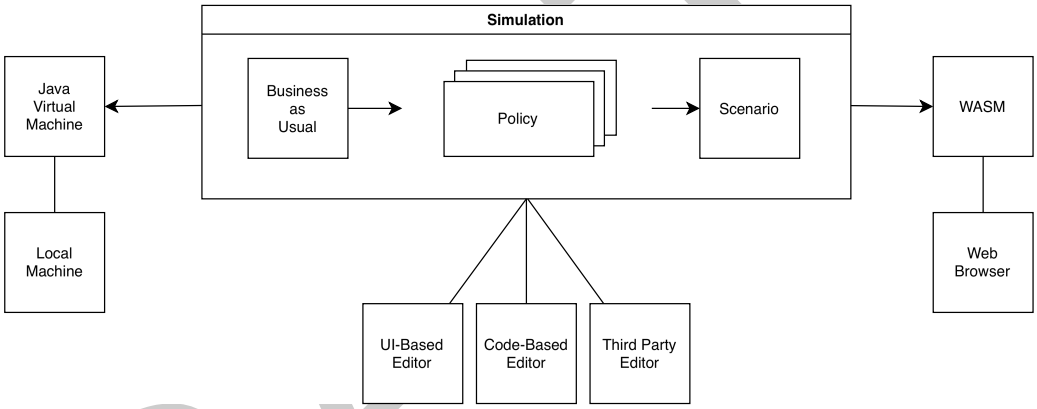

Figure 3: Diagram describing multi-modal execution in which simulations run across different platforms.

Acknowledgments

BSD-licensed. Thanks Amanda Anderson-You, Tina Birmpili, Matt Fisher, Ava Hu, Kevin Koy, Douglas McCauley, Alejandro Ramirez-Pabon, Frederico San Martini, Annie Snyder, Suzanne Spencer, Elina Yuen, runtime dependencies (Ajax.org, 2010; Apache Software Foundation, 2024; Bostock, 2024; Brewer et al., 2013; Chart.js Contributors, 2024; Ferdinandi, 2024; Holt, 2024; Konshin, 2024; OpenJS Foundation, 2024; Parr, 2013; Verou, 2024; Webpack Contributors, 2024), and valued community members listed at https://kigalisim.org/guide/credits.html. Funding from the Eric and Wendy Schmidt Center for Data Science and the Environment at UC Berkeley. No conflicts. AI assistants used with constrained tasks and strict human review (Anthropic, 2025; GitHub et al., 2025; JetBrains, 2025; Replit, 2025). Paper uses drawio (JGraph & draw.io, 2023).

References

Ajax.org. (2010). *Ace: The high performance code editor for the web*. https://ace.c9.io/

Ang, B. W., Goh, T., & Su, B. (2016). Carbon emission intensity in electricity production: A global analysis. *Energy Policy*, *94*, 56–63. https://doi.org/10.1016/j.enpol.2016.03.038

Answer.AI. (2024). *The /llms.txt file*. https://llmstxt.org/

[2]Local assembly can be modeled as domestic production.

[3]A JS-based ANTLR visitor facilitating real-time translation between the UI-editor and the code-based editor.

Anthropic. (2025). *Claude*. https://www.anthropic.com/claude

Apache Software Foundation. (2024). *Apache commons CSV*. https://commons.apache.org/proper/commons-csv/

Appleton, M. (2022). *Programming portals*. https://maggieappleton.com/programming-portals

Augspurger, T., Malliaraki, E., & Hopkins, J. (2023). *Open source in environmental sustainability*.

Birmpili, T. (2018). Montreal protocol at 30: The governance structure, the evolution, and the kigali amendment. *C. R. Geosci.*, *350*(7), 425–431.

Bostock, M. (2024). *D3.js: Data-driven documents*. https://d3js.org/

Brewer, C., Harrower, M., Sheesley, B., Woodruff, A., & Heyman, D. (2013). Colorbrewer 2.0. In *ColorBrewer*. The Pennsylvania State University. https://colorbrewer2.org

Brown, M. (2015). *Super mario 3D world's 4 step level design*. Game Maker's Toolkit. https://youtu.be/dBmIkEvEBtA

Brown, M. (2017). How nintendo solved zelda's open world problem. In *Game Maker's Toolkit*. https://gmtk.substack.com/p/how-nintendo-solved-zeldas-open-world

Chart.js Contributors. (2024). *Chart.js: Simple yet flexible JavaScript charting library*. https://www.chartjs.org/

Ferdinandi, C. (2024). *Tabby: Lightweight JavaScript plugin for managing tabs*. https://github.com/cferdinandi/tabby

Friesen, J. (2024). Math, BigDecimal, and BigInteger. In *Learn java fundamentals: A primer for java development and programming* (pp. 311–324). Apress. https://doi.org/10.1007/979-8-8688-0351-2_12

GitHub, OpenAI, & Microsoft. (2025). *GitHub copilot*. https://github.com/features/copilot

Hayashida, K. (2012). *The structure of fun: Learning from super mario 3D land's director*. https://www.gamedeveloper.com/design/the-structure-of-fun-learning-from-i-super-mario-3d-land-i-s-direc

Holt, M. (2024). *Papa parse: Powerful CSV parser for JavaScript*. https://www.papaparse.com/

JetBrains. (2025). *JetBrains AI assistant*. https://www.jetbrains.com/ai/

JGraph, & draw.io. (2023). Jgraph/drawio: Draw.io is a JavaScript, client-side editor for general diagramming and whiteboarding. In *GitHub*. GitHub, Inc. https://github.com/jgraph/drawio

Konshin, A. (2024). *TeaVM: Build fast, modern web applications in java*. https://teavm.org/

McKenzie, R., Bernhard, G., Liley, B., Disterhoft, P., Rhodes, S., Bais, A., Morgenstern, O., Newman, P., Oman, L., Brogniez, C., & Simic, S. (2019). Success of montreal protocol demonstrated by comparing high-quality UV measurements with "world avoided" calculations from two chemistry-climate models. *Sci. Rep.*, *9*(1), 12332.

MLF Staff. (2024). *Preparation of Kigali HFC implementation plans*. Secretariat of the Multilateral Fund for the Implementation of the Montreal Protocol; Presentation at 46th Meeting of the Open-Ended Working Group of the Parties to the Montreal Protocol. https://ozone.unep.org/system/files/documents/OEWG/%20KIP/%20Side/%20Event/%20MLFS_070824.pdf

Multilateral Fund Secretariat. (2024). *Policies, procedures, guidelines and criteria of the multilateral fund for the implementation of the montreal protocol*. https://www.multilateralfund.org/api/drupal-documents/download/file/4286a8ec-a765-493c-bc0a-2155717410b8?filename=policies_combined.pdf

DRAFT

OpenJS Foundation. (2024). *QUnit: A JavaScript unit testing framework*. https://qunitjs.com/

Parr, T. (2013). *ANTLR (ANother tool for language recognition)*. https://www.antlr.org/

Pottinger, A. S., Biyani, N., Geyer, R., McCauley, D. J., Bruyn, M. de, Morse, M. R., Nathan, N., Koy, K., & Martinez, C. (2023). *Combining game design and data visualization to inform plastics policy: Fostering collaboration between science, decision-makers, and artificial intelligence*. https://arxiv.org/abs/2312.11359

Pottinger, A. S., & Zarpellon, G. (2023). Pyafscgap.org: Open source multi-modal python-based tools for NOAA AFSC RACE GAP. *Journal of Open Source Software*, *8*(86), 5593. https://doi.org/10.21105/joss.05593

Pottinger, A., Connor, L., Guzder-Williams, B., Weltman-Fahs, M., Gondek, N., & Bowles, T. (2025). Climate-driven doubling of u.s. Maize loss probability: Interactive simulation with neural network monte carlo. *Journal of Data Science, Statistics, and Visualisation*, *5*(3). https://doi.org/10.52933/jdssv.v5i3.134

Replit. (2025). *Replit AI agent*. https://replit.com/ai

Schindler, J., & Cabot, J. (2022). An empirical evaluation of a novel domain-specific language—modelling vehicle routing problems with athos. *Empirical Software Engineering*, *27*(7), 1–44. https://doi.org/10.1007/s10664-022-10210-w

Skills, G. D. (2020). *Core loops in gameplay*. https://gamedesignskills.com/game-design/core-loops-in-gameplay/

UNEP OzonAction. (n.d.). *National ozone officers' capacity building*. United Nations Environment Programme; Web page. https://www.unep.org/ozonaction/national-ozone-officers-capacity-building

United Nations. (1987). *Montreal protocol on substances that deplete the ozone layer, article 5: Special situation of developing countries*. https://ozone.unep.org/treaties/montreal-protocol/articles/article-5-special-situation-developing-countries

Velders, G. J. M., Fahey, D. W., Daniel, J. S., McFarland, M., & Andersen, S. O. (2009). The large contribution of projected HFC emissions to future climate forcing. *Proc. Natl. Acad. Sci. U. S. A.*, *106*(27), 10949–10954.

Verou, L. (2024). *Prism: Lightweight, extensible syntax highlighter*. https://prismjs.com/

Webpack Contributors. (2024). *Webpack: A static module bundler for modern JavaScript applications*. https://webpack.js.org/

Wheeler, K. (2004). *HyperTalk: The language for the rest of us*. https://www.memoryhole.net/~kyle/papers/hypertalk.pdf

World Meteorological Organization. (2022). *Scientific assessment of ozone depletion: 2022* (p. 509). GAW Report No. 278. ISBN: 978-9914-733-97-6

DRAFT